



TD1-Partie 1- Ar

الأهداف :

1. فهم أساسيات هندسة البرمجيات ودورة حياة تطوير البرمجيات.
2. ممارسة تحديد المتطلبات.
3. البدء في نمذجة حالات الاستخدام.
4. استكشاف أساسيات إدارة الإصدارات.

التمرين 1: مفاهيم ومصطلحات
السؤال 1: عرف المصطلحات التالية:

- هندسة البرمجيات، البرمجيات، البرامج الجاهزة، الهندسة المعمارية البرمجية، دورة حياة تطوير البرمجيات، UML، التخطيط.

السؤال 2: ما الفرق بين: اللغة، الأسلوب، والبراديجم (قدم أمثلة).

السؤال 3: ما هي "إدارة المشاريع البرمجية" بشكل عام؟

السؤال 4: ما هي أفضل تقنيات وأساليب هندسة البرمجيات؟

السؤال 5: ما هي التحديات الرئيسية التي تواجه هندسة البرمجيات؟

السؤال 6: ما هي تكاليف هندسة البرمجيات؟

السؤال 7: ما هي التغييرات التي أحدثتها الإنترنت في هندسة البرمجيات؟

السؤال 8: ما هي الصفات الأساسية المطلوبة في كراسة الشروط لمشروع برمجي، وكذلك الأسباب الرئيسية لنجاح أو فشل المشروع، وكيف يمكن تحسين عوامل الجودة المتعلقة بالتصحيح (الفائدة) وسهولة الاستخدام؟

السؤال 9: كيف يمكن تقليل الفجوة بين الاحتياجات الحقيقية للعميل والاحتياجات المعبر عنها في كراسة الشروط؟

السؤال 10: ما الفرق بين "دورة حياة" و "دورة تطوير" البرمجيات؟

السؤال 11: ما هي النماذج الرئيسية لدورة حياة البرمجيات؟

السؤال 12: تصف نماذج دورة الحياة الكلاسيكية أو الحديثة جميعها منهجية إنتاج البرمجيات عالية الجودة. ومع ذلك، توجد سمة تميزها. ما هي؟ وما هي حدود النماذج الكلاسيكية؟ وما هي مزايا الأساليب الحديثة مثل Agile و DevOps؟

السؤال 13: قارن بين النماذج الأولية المؤقتة وغير المؤقتة.

السؤال 14: كيف يتم وصف أنشطة عملية التطوير؟

السؤال 15: ما هي مراحل تطوير البرمجيات؟



TD1-Partie 1- EN

Objectives:

1. Understand the basics of software engineering and the software development lifecycle.
2. Practise requirement identification.
3. Get started with use case modelling.
4. Explore the basics of version management.

Exercise 1: Concepts and Terminology

Q1: Define the following terms:

- Software Engineering, Software, Software Package, Software Architecture, Software Development Lifecycle, UML, Planning.

Q2: What is the difference between language, method, and paradigm (give examples).

Q3: What does "Software Project Management" generally entail?

Q4: What are the best software engineering techniques and methods?

Q5: What are the key challenges facing software engineering?

Q6: What are the costs of software engineering?

Q7: What differences has the Internet made to software engineering?

Q8: What essential qualities are required in a software project's specification document, along with the main reasons for a project's success or failure, and how can quality factors related to correction (utility) and usability be improved?

Q9: How can the gap between a client's actual needs and those expressed in the specification document be reduced?

Q10: What is the distinction between "software lifecycle" and "software development cycle"?

Q11: What are the main models of a Software Lifecycle?

Q12: Classical and modern lifecycle models all outline the approach for producing quality software, but one characteristic differentiates them. What is it, and what are the limitations of classical models? What are the benefits of modern methods such as Agile and DevOps?

Q13: Compare disposable prototyping to non-disposable prototyping.

Q14: How are the activities of a development process described?

Q15: What are the stages of software development?

TD1-Partie 1- Solution-Ar

الأهداف :

5. فهم أساسيات هندسة البرمجيات ودورة حياة تطوير البرمجيات.
6. ممارسة تحديد المتطلبات.
7. البدء في نمذجة حالات الاستخدام.
8. استكشاف أساسيات إدارة الإصدارات.

التمرين 1: مفاهيم ومصطلحات
السؤال 1: عرف المصطلحات التالية:

- هندسة البرمجيات، البرمجيات، البرامج الجاهزة، الهندسة المعمارية البرمجية، دورة حياة تطوير البرمجيات، UML، التخطيط.

حل السؤال 1:

- **هندسة البرمجيات:** هي تخصص يطبق مبادئ علمية ورياضية وتقنية لتطوير وصيانة البرمجيات. يهدف إلى إنتاج برمجيات عالية الجودة، موثوقة وقابلة للصيانة، مع تحسين الموارد والوقت. استخدم هذا المصطلح لأول مرة في مؤتمر الناتو حول أزمة البرمجيات عام 1968، ولكن يعود الفضل في هذا المفهوم إلى عالمة مارجريت هاميلتون، التي صممت النظام المدمج لبرنامج أبولو. يشمل هندسة البرمجيات مجموعة من الأساليب والتقنيات والأدوات التي تساعد في إنتاج برمجيات بجودة عالية مع التحكم في التكاليف والمواعيد النهائية.
- **البرمجيات:** مجموعة مكونة من برنامج أو عدة برامج وإجراءات، إضافة إلى الملفات اللازمة لجعلها قابلة للتشغيل (وثائق التثبيت والاستخدام والتطوير والصيانة). يمكن أن تكون البرمجيات نظام تشغيل، تطبيق، أو مكتبة وظائف.
- **البرامج الجاهزة (Progiciel):** هي برمجيات معيارية مصممة لتلبية احتياجات مشتركة ويمكن استخدامها من قبل عدد كبير من الشركات أو المستخدمين دون تعديل. على سبيل المثال، أنظمة تخطيط موارد المؤسسة (ERP) هي برامج جاهزة تغطي احتياجات إدارة الشركات.
- **الهندسة المعمارية البرمجية:** هي بنية أو تصميم عالي المستوى للبرمجيات، حيث تحدد كيفية تفاعل الأجزاء المختلفة للنظام (الوحدات، المكونات) مع بعضها البعض ومع البيئة الخارجية، لضمان الجودة، سهولة الصيانة، وقابلية التوسع للنظام.
- **دورة حياة تطوير البرمجيات:** هي مجموعة من المراحل التي يمر بها البرنامج بدءاً من التصميم إلى النشر والصيانة. تتضمن المراحل الرئيسية جمع المتطلبات، التصميم، التطوير، الاختبار، النشر، والصيانة.
- **لغة النمذجة الموحدة (UML):** هي لغة نمذجة بصرية تُستخدم لتمثيل تصميم النظام البرمجي بشكل رسومي. تتيح UML إنشاء أنواع مختلفة من المخططات (حالات الاستخدام، الفئات، التسلسل، إلخ) لتوثيق الجوانب الهيكلية والسلوكية للبرمجيات.
- **التخطيط:** في إدارة المشاريع البرمجية، التخطيط هو المرحلة التي تشمل تنظيم المهام، تحديد المواعيد النهائية، تخصيص الموارد، وتقدير الميزانية لضمان إتمام المشروع في الوقت المحدد ووفقاً للموارد المتاحة.

السؤال 2: ما الفرق بين: اللغة، الأسلوب، والبراديجم (قدم أمثلة).

حل السؤال 2:

الأسلوب = اللغة + المنهج.

TD1-Partie 1- Solution-Ar

التعريف	أمثلة
اللغة: لغة البرمجة هي مجموعة من القواعد والتركيبات التي تمكن المطورين من كتابة تعليمات يمكن أن يفهمها وينفذها الكمبيوتر. لكل لغة خصائصها وتركيباتها ومجالات التطبيق الخاصة بها.	Python: لغة عالية المستوى شائعة لتطوير تطبيقات الويب وتحليل البيانات والذكاء الاصطناعي. Java: معروفة بكونها محمولة وتستخدم في تطبيقات الأعمال وتطبيقات أندرويد. C++: تستخدم للتطبيقات منخفضة المستوى والألعاب، وتتميز بالكفاءة والتحكم في الذاكرة.
الأسلوب: في هندسة البرمجيات، الأسلوب هو مجموعة من التقنيات والعمليات والممارسات التي توجه تطوير البرمجيات، بدءًا من التخطيط إلى الصيانة. وهو العملية التي تنظم طريقة إدارة تطوير البرمجيات.	الأسلوب الرشيق (Agile): نهج تكراري ومرن لتطوير البرمجيات يشجع التعاون بين الفرق وأصحاب المصلحة. يعتمد على دورات تطوير قصيرة (سبرنت) تسمح بتحسينات مستمرة وتعديلات وفقًا لاحتياجات العميل. يحتوي على أساليب مثل سكروم وكنبان. أسلوب الشلال (Waterfall): نموذج تطوير تسلسلي حيث يجب إكمال كل مرحلة (متطلبات، تصميم، تطوير، اختبار، صيانة) قبل الانتقال إلى المرحلة التالية. مناسب للمشاريع ذات المتطلبات المحددة مسبقًا. أسلوب ميريز (MERISE): أسلوب لتحليل وتصميم نظم المعلومات. يعتمد على نهج منظم ويفصل بين البيانات والمعالجات لنمذجة احتياجات النظام.
Paradigm البرمجي هو أسلوب أو فلسفة (مثل البرمجة الكائنية أو الوظيفية) التي تؤثر على تنظيم وبنية الكود. تحدد البراديجمات كيفية التعامل مع المشاكل في لغة معينة وتؤثر على اختيار الهياكل والطرق والخوارزميات.	(Imperative): يصف البرنامج تسلسلاً من التعليمات لتغيير حالة النظام (مثل لغة C و Python). (Object-Oriented): ينظم الكود في كائنات تحتوي على بيانات وطرق للتفاعل مع هذه البيانات (مثل Java و Python و C++). وظيفي (Functional): يفضل استخدام الدوال بدون تأثيرات جانبية وتعابير رياضية (مثل Haskell و Lisp و JavaScript جزئيًا). منطقي (Logical): يعتمد على المنطق الرسمي، ويصف العلاقات بين الحقائق والقواعد مثل (Prolog).

السؤال 3: ما هي "إدارة المشاريع البرمجية" بشكل عام؟

حل السؤال 3: تتضمن إدارة المشاريع البرمجية التخطيط والتنظيم والمتابعة وتنسيق الأنشطة لتطوير برمجيات ذات جودة عالية، مع احترام المواعيد النهائية والميزانية.

الخطوات الرئيسية:

1. **تحديد الأهداف:** جمع وتحليل المتطلبات الوظيفية وغير الوظيفية.
2. **التخطيط:** إعداد خطة تتضمن المهام، الجدول الزمني، والميزانية.
3. **إدارة الموارد:** تحديد الأدوار وتنظيم الأدوات والموارد.
4. **المتابعة والتحكم:** مراقبة التقدم وإجراء التعديلات حسب الحاجة وإدارة المخاطر.
5. **ضمان الجودة:** إجراء الاختبارات لضمان الموثوقية والأداء.

TD1-Partie 1- Solution-Ar

6. التسليم والنشر: إعداد البرنامج ليتم استخدامه في بيئة الإنتاج.
7. الصيانة: توفير الدعم والتحديثات.

السؤال 4: ما هي أفضل تقنيات وأساليب هندسة البرمجيات؟

حل السؤال 4: يجب أن تُدار جميع المشاريع البرمجية بشكل احترافي، ولكن تتناسب التقنيات المختلفة مع أنواع الأنظمة المختلفة. على سبيل المثال، يجب تطوير الألعاب باستخدام سلسلة من النماذج الأولية، في حين أن أنظمة التحكم الحرجة تتطلب مواصفات كاملة وقابلة للتحليل. لا توجد تقنيات وأساليب تناسب جميع الأنظمة.

السؤال 5: ما هي التحديات الرئيسية التي تواجه هندسة البرمجيات؟

حل السؤال 5: التكيف مع التنوع المتزايد، متطلبات تقليل أوقات التسليم، وتطوير برمجيات موثوقة.

السؤال 6: ما هي تكاليف هندسة البرمجيات؟

حل السؤال 6: تقريباً 60% من تكاليف البرمجيات تتعلق بالتطوير، و40% تتعلق بالاختبار. في البرمجيات المخصصة، غالباً ما تتجاوز تكاليف التطوير تكاليف الصيانة.

السؤال 7: ما هي التغييرات التي أحدثها الإنترنت في هندسة البرمجيات؟

حل السؤال 7: لم يؤد الإنترنت إلى تطوير أنظمة ضخمة وموزعة تعتمد على الخدمات فحسب، بل دعم أيضاً إنشاء صناعة "التطبيقات" للأجهزة المحمولة، مما غير من اقتصاديات البرمجيات.

السؤال 8: ما هي الصفات الأساسية المطلوبة في كراسة الشروط لمشروع برمجي، وكذلك الأسباب الرئيسية لنجاح أو فشل المشروع، وكيف يمكن تحسين عوامل الجودة المتعلقة بالتصحيح (الفائدة) وسهولة الاستخدام؟

حل السؤال 8:

- يجب أن تكون كراسة الشروط:
 - واضحة ودقيقة: تحديد احتياجات ومتطلبات المشروع.
 - شاملة: تغطي جميع الجوانب الأساسية (الوظائف، القيود، المواعيد).
 - مطابقة للتوقعات: تلبي احتياجات المستخدمين الفعلية.
 - قابلة للتطور: يمكن تعديلها حسب المتطلبات الجديدة.
- أسباب نجاح أو فشل المشروع: السبب الرئيسي وراء معظم المشاكل هو سوء فهم طبيعة المشكلات التي يجب حلها، وخاصةً الاستهانة الشديدة بصعوبتها.

أسباب فشل المشروع	أسباب نجاح المشروع
المواصفات غير المكتملة أو الغامضة	المشاركة الفعالة للمستخدمين
ضعف التواصل بين أصحاب المصلحة	دعم والتزام الإدارة
سوء إدارة التغييرات	تعريف واضح ودقيق لاحتياجات العميل
الاستهانة بتعقيد المشروع	التخطيط الفعال مع مراقبة المواعيد والميزانية
نقص مشاركة الفريق أو المستخدمين	التواصل المستمر بين الفرق وأصحاب المصلحة

TD1-Partie 1- Solution-Ar

الإدارة الاستباقية للمخاطر وتوقع العقبات	غياب إدارة المخاطر
حافزية وكفاءة الفريق	عدم التكيف مع المتطلبات المتغيرة
القدرة على التكيف مع التغييرات	مقاومة التعديلات الضرورية

- العوامل الرئيسية لجودة البرمجيات:
 - ✓ تتضمن عوامل جودة البرمجيات الرئيسية الموثوقية، وقابلية الصيانة، وسهولة الاستخدام، والكفاءة، وقابلية التوسع.
 - ✓ الصحة: تحقيق الوظائف المحددة.
 - ✓ الموثوقية: العمل بشكل صحيح في الظروف غير العادية.
 - ✓ سهولة الاستخدام: واجهة بديهية وسهلة الاستخدام.
 - ✓ التوافق: التفاعل السلس مع الأنظمة الأخرى.
 - ✓ الكفاءة: الاستخدام الأمثل للموارد.
 - ✓ قابلية النقل: العمل على منصات مختلفة.
 - ✓ إمكانية إعادة الاستخدام: استخدام الكود في مشاريع أخرى.
 - ✓ قابلية الصيانة: سهولة التعديل والتحسين.
 - ✓ قابلية التوسع: القدرة على التكيف مع التغييرات في المواصفات.
 - ✓ النزاهة: حماية البيانات والكود.
 - ✓ الدقة الزمنية: التسليم في الوقت المحدد.

تحسين عوامل الجودة

- التصحيح (الفائدة):
 - إجراء تحليل شامل للاحتياجات من البداية.
 - تحسين التواصل (لغة مشتركة ونهج تشاركي).
 - العمل بمنهجية ودقة لتقليل الأخطاء.
- سهولة الاستخدام:
 - تحليل طرق تشغيل المستخدمين لتكييف البرنامج.
 - تصميم واجهة مستخدم (IHM) مريحة لتسهيل التنقل والاستخدام.

السؤال 9: كيف يمكن تقليل الفجوة بين الاحتياجات الحقيقية للعميل والاحتياجات المعبر عنها في كراسة الشروط؟

حل السؤال 9:

- التحليل المتعمق: تنظيم ورش العمل والمقابلات لتوضيح الاحتياجات.
- مشاركة المستخدمين النهائيين: إشراك المستخدمين من البداية لفهم توقعاتهم بشكل أفضل.
- التحقق المستمر: استخدام النماذج الأولية والتحقق من كل خطوة مع العميل.
- التواصل المنتظم: الحفاظ على تواصل واضح وتجنب المصطلحات التقنية.
- الأساليب الرشيقية: تبني دورات تطوير قصيرة لدمج ملاحظات العميل وتعديل المواصفات.

السؤال 10: ما الفرق بين "دورة حياة" و"دورة تطوير" البرمجيات؟

حل السؤال 10: تشمل دورة حياة البرمجيات جميع المراحل، من التصميم الأولي حتى نهاية عمر البرمجيات. تشمل مراحل مثل تحليل المتطلبات، التصميم، التطوير، الصيانة، والإيقاف. الهدف الرئيسي لهذه الدورة هو ضمان جودة وتطوير البرمجيات على مدار حياتها. بينما تركز دورة تطوير البرمجيات على إنشاء إصدار أولي أو تحديث للبرنامج. تشمل مراحل مثل التحليل، التصميم، الترميز،

TD1-Partie 1- Solution-Ar

الاختبار، والتسليم، بهدف إنتاج إصدار وظيفي. لذا، فإن دورة التطوير جزء من دورة الحياة الأشمل، التي تتضمن أيضاً الصيانة والتطوير.

السؤال 11: ما هي النماذج الرئيسية لدورة حياة البرمجيات؟

الحل: Q11:

النموذج	الخصائص	المزايا	العيوب
النموذج الشلالي	مراحل متسلسلة: التصميم، التطوير، الاختبار، النشر، الصيانة.	بسيط وسهل الفهم والإدارة.	صعب التراجع بعد إنهاء المرحلة.
النموذج التكراري	تطوير على شكل دورات مع مراجعات وتحسينات تدريجية.	مرونة لإدماج التغييرات.	قد يكون صعب الإدارة إذا لم يتم تحديد التكرارات.
النموذج الرشيق (AGILE)	تطوير على خطوات قصيرة مع دورات وجلسات مراجعة متكررة.	استجابة سريعة للتغييرات، تعاون قوي.	يتطلب تواصل مستمر ومشاركة من الأطراف المعنية.
النموذج في V	مشابه للشلالي، ولكن مع مراحل اختبار موازية.	اختبار منظم وتحقق في كل مرحلة.	معقد في إدارة مراحل الاختبار والتطوير.
النموذج الحلزوني	مزيج من التطوير التكراري وإدارة المخاطر مع مراحل تخطيطية.	يركز على إدارة المخاطر ومرونة التغييرات.	قد يكون مكلفاً من حيث الوقت والموارد.
نموذج DEVOPS	دمج التطوير والعمليات لتحسين دورة التسليم.	نشرات متكررة واستجابة سريعة للاحتياجات.	قد يتطلب تغييرات ثقافية كبيرة في الفريق.

السؤال 12: تصف نماذج دورة الحياة الكلاسيكية أو الحديثة جميعها منهجية إنتاج البرمجيات عالية الجودة. ومع ذلك، توجد سمة تميزها. ما هي؟ وما هي حدود النماذج الكلاسيكية؟ وما هي مزايا الأساليب الحديثة مثل Agile و DevOps؟

الحل: Q12: تصف النماذج الكلاسيكية والحديثة منهجيات إنتاج البرمجيات، لكنها تختلف أساساً في المرونة تجاه التغييرات. النماذج الكلاسيكية، مثل النموذج الشلالي، تكون صارمة وتتبع بنية متسلسلة حيث يجب إكمال كل خطوة قبل الانتقال إلى التالية. بينما تفضل الأساليب الحديثة، مثل Agile و DevOps، عمليات تكرارية وتكيفية، مما يسمح بإدخال التعديلات أثناء المشروع.

حدود النماذج الكلاسيكية:

1. الصرامة: يصعب العودة إلى مراحل سابقة عند الحاجة إلى تغييرات.
2. نقص المرونة: التعديلات بعد تحديد المواصفات تكون معقدة.
3. التأخير في التسليم: تستغرق النسخة القابلة للاستخدام وقتاً طويلاً، مما قد يؤدي إلى خيبة أمل العميل.
4. ضعف التفاعل مع العميل: المشاركة القليلة للعميل يمكن أن تؤدي إلى فجوة بين المنتج النهائي والتوقعات.

مزايا الأساليب الحديثة (Agile و DevOps):

1. التكيف والتكرار: تتيح الدورات القصيرة تعديلات سريعة بناءً على ملاحظات العميل.
2. التسليم المستمر: تُسلم البرمجيات بشكل مستمر، مما يسمح بتحسينات سريعة.
3. التعاون المكثف: تفاعل قوي بين الفرق (التطوير، العمليات، العميل) لتحسين الفهم.
4. تحسين الجودة: بفضل الاختبارات المستمرة والمتابعة في الوقت الفعلي.

TD1-Partie 1- Solution-Ar

السؤال 13: قارن بين النماذج الأولية المؤقتة وغير المؤقتة.

الحل: Q13:

الجانب	النموذج الأولي المؤقت	النموذج الأولي غير المؤقت
الهدف	اختبار الأفكار بسرعة دون نية إعادة الاستخدام.	إنشاء نسخة أولية تتطور لتصبح المنتج النهائي.
الاستخدام	مصمم ليتم التخلص منه بعد التحقق من المتطلبات.	مصمم ليتم تحسينه وإدماجه في البرمجيات النهائية.
جودة الكود	غالبًا يكون أقل جودة، يركز على السرعة.	كود بجودة عالية ويتوافق مع معايير التطوير.
وقت التطوير	تطوير سريع لأنه مؤقت.	قد يستغرق وقتًا أطول لأنه مصمم ليكون دائمًا.
الميزة	يتيح دورات سريعة وسهلة التنفيذ.	توفير الموارد نظرًا لإعادة استخدام الكود.
العيب	يتطلب إعادة التطوير للمنتج النهائي.	يتطلب تخطيط وتطوير مبدئي أكثر دقة.

السؤال 14: كيف يتم وصف أنشطة عملية التطوير؟

الحل: يتم وصف أنشطة عملية التطوير من خلال:

- المدخلات (المادة الخام)؛
- المخرجات (النتيجة)؛
- المشاركين والأدوار (من؟)؛
- وصف النشاط (ما المشكلة المراد معالجتها؟)؛
- المعايير والإرشادات وأفضل الممارسات (كيف؟).

السؤال 15: ما هي مراحل تطوير البرمجيات؟

الحل: المراحل الأساسية لتطوير البرمجيات:

1. تحليل المتطلبات: تحديد وتوثيق احتياجات المستخدمين والقيود.
2. المواصفات: توثيق الوظائف والمتطلبات التقنية.
3. التصميم: تخطيط هيكل البرمجيات لتنظيم مكوناتها.
4. التطوير: برمجة الوظائف وفقًا للمواصفات، مع اختبارات وحدات.
5. الاختبارات والتحقق: ضمان تطابق البرمجيات مع المواصفات.
6. النشر: تركيب البرمجيات للمستخدمين النهائيين مع تدريب إذا لزم.
7. الصيانة: تصحيح الأخطاء وتحديث البرمجيات لتلبية الاحتياجات المتغيرة.

**TD1-Partie 1- Solution-En****Objectives:**

5. Understand the basics of software engineering and the software development lifecycle.
6. Practise requirement identification.
7. Get started with use case modelling.
8. Explore the basics of version management.

Exercise 1: Concepts and Terminology

Q1: Define the following terms:

- Software Engineering, Software, Software Package, Software Architecture, Software Development Lifecycle, UML, Planning.

Solution Q1:

- **Software Engineering:** This is a discipline that applies scientific, mathematical, and technical principles to the development and maintenance of software. Its aim is to produce high-quality, reliable, and maintainable software while optimising resources and time. The term was first used at the NATO conference on the software crisis in 1968, although the concept itself is credited to Margaret Hamilton, who designed the embedded system for the Apollo programme. Software engineering encompasses methods, techniques, and tools that contribute to producing quality software with controlled costs and schedules.
- **Software:** A set consisting of one or more programs, procedures, and the necessary files to make them operational (installation, usage, development, and maintenance documents). Software can be an operating system, an application, or a library of functions.
- **Software Package (Progiciel):** This is a standardised software designed to meet common needs, intended to be used by numerous companies or users without specific modifications. For example, Enterprise Resource Planning (ERP) systems are software packages that cover companies' management needs.
- **Software Architecture:** This is the high-level structure or design of software, defining how different parts of a system (modules, components) interact with each other and the external environment, ensuring the quality, maintainability, and scalability of the system.
- **Software Development Lifecycle:** This is the series of phases a software undergoes from design to deployment and maintenance. Key phases include requirements gathering, design, development, testing, deployment, and maintenance.
- **Unified Modelling Language (UML):** A visual modelling language used to graphically represent the design of a software system. UML allows for the creation of various types of diagrams (use cases, classes, sequences, etc.) to document the structural and behavioural aspects of software.
- **Planning:** In software project management, planning is the stage involving task organisation, setting deadlines, allocating resources, and estimating the budget to ensure project completion on time and within available resources.



TD1-Partie 1- Solution-En

Q2: What is the difference between language, method, and paradigm (give examples).

Solution Q2:

Method = Language + Approach.

Definition	Examples
Language : a programming language is a set of rules and syntax that allows developers to write instructions that a computer can understand and execute. Each language has its own characteristics, syntax, and specific application domains.	Python: a high-level language popular for web development, data analysis, and artificial intelligence. Java: known for its portability and use in enterprise applications and android apps. C++: used for low-level applications and games, valued for efficiency and memory control.
Method : in software engineering, a method is a set of techniques, processes, and practices that guide software development, from planning to maintenance. It structures the way software development should be managed.	Agile method : an iterative and flexible software development approach that fosters collaboration between teams and stakeholders. It relies on short development cycles (sprints) for continuous improvements and adjustments based on client needs, including specific methods like scrum and kanban. Waterfall method : a sequential development model where each phase (requirements, design, development, testing, maintenance) must be completed before moving to the next, suitable for projects with clearly defined requirements from the start. Merise method : an information systems analysis and design method, based on a structured approach that separates data and processing to model system needs.
Paradigm : a programming paradigm is a style or philosophy (such as object-oriented or functional) that influences the organisation and structure of code. Paradigms dictate how problems should be addressed in a given language and affect the choice of data structures, techniques, and algorithms.	Imperative paradigm: the program describes a sequence of instructions to change the system state (e.g., c, python). Object-oriented paradigm: organises code into objects containing data and methods to interact with that data (e.g., java, python, c++). Functional paradigm: emphasises using functions without side effects and mathematical expressions (e.g., haskell, lisp, and partly javascript with pure functions). Logical paradigm: based on formal logic, it



TD1-Partie 1- Solution-En

	describes relationships between facts and rules (e.g., prolog).
--	---

Q3: What does "Software Project Management" generally entail?

Solution Q3: Software project management involves planning, organising, tracking, and coordinating activities to develop high-quality software, respecting deadlines and budgets.

Main steps:

1. **Objective definition:** Gather and analyse functional and non-functional requirements.
2. **Planning:** Establish a task schedule, timeline, and budget.
3. **Resource management:** Assign roles and organise tools and resources.
4. **Monitoring and control:** Track progress, make adjustments as needed, and manage risks.
5. **Quality assurance:** Perform tests to ensure reliability and performance.
6. **Delivery and deployment:** Prepare the software for production.
7. **Maintenance:** Provide support and updates.

Q4: What are the best software engineering techniques and methods?

Solution Q4: While all software projects should be professionally managed, different techniques suit different types of systems. For instance, games should be developed using a series of prototypes, whereas safety-critical systems require complete and analyzable specifications. There are no techniques and methods suitable for everything.

Q5: What are the key challenges facing software engineering?

Solution Q5: Coping with increasing diversity, demands for reduced delivery times, and developing trustworthy software.

Q6: What are the costs of software engineering?

Solution Q6: Approximately 60% of software costs are development costs, and 40% are testing costs. For custom software, evolution costs often exceed development costs.

Q7: What differences has the Internet made to software engineering?

Solution Q7: The Internet has led to the development of massive, distributed, service-based systems and has supported the creation of an app industry for mobile devices, changing software economics.



TD1-Partie 1- Solution-En

Q8: What essential qualities are required in a software project's specification document, along with the main reasons for a project's success or failure, and how can quality factors related to correction (utility) and usability be improved?

Solution Q8:

- Specification document should be:
 - **Clear and precise:** Outlines project needs and requirements.
 - **Comprehensive:** Covers all essential aspects (features, constraints, deadlines).
 - **Aligned with expectations:** Meets actual user needs.
 - **Evolvable:** Can be modified according to new requirements.
- **Reasons for Project Success or Failure:** The primary reason behind most issues is a poor understanding of the problems to be resolved and, more critically, a serious underestimation of their complexity.

Reasons for project success	Reasons for project failure
Active user involvement	Incomplete or ambiguous specifications
Management support and commitment	Weak communication among stakeholders
Clear and precise definition of client needs	Poor change management
Effective planning with tracking of timelines and budget	Underestimation of project complexity
Constant communication between teams and stakeholders	Lack of team or user involvement
Proactive risk management and anticipation of obstacles	Absence of risk management
Team motivation and skills	Failure to adapt to evolving requirements
Adaptability to changes	Resistance to necessary modifications

- **Key Quality Factors of Software:**
Key software quality factors include reliability, maintainability, usability, efficiency, and scalability.
 - ✓ **Validity :** Meeting the specified functions.
 - ✓ **Reliability :** Correct operation under abnormal conditions.
 - ✓ **Usability :** Intuitive interface and ease of use.
 - ✓ **Compatibility :** Smooth interaction with other systems.
 - ✓ **Efficiency :** Optimal use of resources.
 - ✓ **Portability :** Ability to operate on different platforms.
 - ✓ **Reusability :** Use of code in other projects.
 - ✓ **Maintainability :** Ease of modification and enhancement.



TD1-Partie 1- Solution-En

- ✓ **Scalability** : Adaptability to specification changes.
- ✓ **Integrity** : Protection of data and code.
- ✓ **Timeliness** : On-time delivery.
- **Improving Quality Factors**
 - **Correction (Utility):**
 - Conduct a thorough needs analysis upfront.
 - Improve communication (common language, participatory approach).
 - Work methodically and rigorously to minimise errors.
 - **Usability:**
 - Analyse user workflows to tailor the software.
 - Design an ergonomic user interface (UI) to facilitate navigation and usage.

Q9: How can the gap between a client's actual needs and those expressed in the specification document be reduced?

Solution Q9:

- **In-depth analysis** : Organise workshops and interviews to clarify requirements.
- **Involve end users** : Engage users from the beginning to better understand their expectations.
- **Continuous validation** : Use prototypes and validate each stage with the client.
- **Regular communication** : Maintain clear communication and avoid technical jargon.
- **Agile methods** : Adopt short cycles to integrate client feedback and adjust specifications.

Q10: What is the distinction between "software lifecycle" and "software development cycle"?

Solution Q10: The software lifecycle encompasses all stages, from initial design to the end of the software's life. It includes phases such as requirements analysis, design, development, maintenance, and retirement, with the primary goal of ensuring the software's quality and evolution throughout its life.

In contrast, the software development cycle focuses specifically on creating an initial version or update of the software. It involves stages such as analysis, design, coding, testing, and delivery, aimed at producing a functional version. Therefore, the development cycle is part of the broader lifecycle, which also includes maintenance and evolution.

Q11: What are the main models of a Software Lifecycle?

Solution Q11:

Model	Characteristics	Advantages	Disadvantages
Waterfall model	Sequential phases: design, development, testing,	Simple to understand and manage.	Rigid, difficult to revert once a phase is



TD1-Partie 1- Solution-En

	deployment, maintenance.		completed.
Iterative model	Development in cycles, allowing for gradual revisions and improvements.	Flexibility to incorporate changes.	Can be hard to manage if iterations are not well defined.
Agile model	Small step development with short cycles and frequent feedback.	Fast response to changes, strong collaboration.	Requires constant communication and stakeholder involvement.
V-model	Similar to the waterfall model but with parallel testing phases.	Systematic testing and validation at each phase.	Complex management of testing and development phases.
Spiral model	Combination of iterative development and risk management with planning phases.	Risk-focused approach, flexible for changes.	Can be costly in terms of time and resources.
Devops model	Integration of development and operations to improve delivery cycles.	More frequent deployments and responsiveness to needs.	May require significant cultural changes within the team.

Q12: Classical and modern lifecycle models all outline the approach for producing quality software, but one characteristic differentiates them. What is it, and what are the limitations of classical models? What are the benefits of modern methods such as Agile and DevOps?

Solution Q12: Classical and modern lifecycle models describe software production approaches, but they mainly differ in **flexibility towards changes**. Classical models, like the waterfall model, are rigid and follow a sequential structure where each stage must be completed before moving to the next. In contrast, modern methods like Agile and DevOps emphasize iterative and adaptive processes, allowing for changes throughout the project.

Limitations of Classical Lifecycle Models:

1. **Rigidity:** Classical models make it hard to revisit previous stages if changes are needed.
2. **Lack of flexibility for unforeseen events:** Once specifications are set, adjustments are complex.
3. **Long delivery timeline:** Delivering a usable version often takes a long time, possibly disappointing clients.
4. **Limited client interaction:** Minimal client involvement can lead to a gap between the final product and real expectations.

Benefits of Modern Methods (Agile and DevOps):



TD1-Partie 1- Solution-En

1. **Adaptability and iteration:** Short development cycles enable quick adjustments based on client feedback.
2. **Continuous delivery:** In DevOps, software is delivered and deployed continuously, allowing for rapid improvements.
3. **Increased collaboration:** Strong interaction between teams (development, operations, and clients) improves understanding of needs.
4. **Enhanced quality:** With frequent testing and real-time tracking, defects are detected and corrected early.

Q13: Compare disposable prototyping to non-disposable prototyping.

Solution Q13:

Aspect	Disposable prototyping	Non-disposable prototyping
Objective	Quickly test ideas or concepts without reuse intentions.	Create an initial version that evolves into the final product.
Usage	Designed to be discarded after requirement validation.	Designed to be improved and integrated into the final software.
Code quality	Often lower quality, focused on speed of creation.	High-quality code that meets development standards.
Development time	Fast development as the prototype is temporary.	May take longer as the prototype is built to last.
Advantage	Allows rapid iterations, easy to create.	Resource-saving since the code is reusable.
Disadvantage	Requires redevelopment for the final product.	Needs more rigorous initial planning and development.

Q14: How are the activities of a development process described?

Solution: Development process activities are described in terms of: a. Activity inputs (raw materials); b. Activity outputs (result); c. Stakeholders and roles (who?); d. Activity description (what problem is being addressed?); e. Standards, guidelines, and best practices (how?).

Q16: What are the stages of software development?

Solution: Key stages of software development:

1. **Requirements analysis:** Identification and documentation of user expectations and constraints.
2. **Specifications:** Formalisation of functionalities and technical requirements in a reference document.
3. **Design:** Planning the software architecture to structure its components.



TD1-Partie 1- Solution-En

4. **Development:** Programming features according to specifications, including unit tests.
5. **Testing and validation:** Ensuring software complies with specifications and functions correctly.
6. **Deployment:** Installing the software for end users with possible training.
7. **Maintenance:** Fixes and updates to adapt the software to evolving needs.

**TD1-Partie 1- Solution-fr****Objectifs :**

1. Comprendre les bases du génie logiciel et le cycle de vie du développement logiciel.
2. Pratiquer l'identification des exigences.
3. S'initier à la modélisation de cas d'utilisation.
4. Explorer les bases de la gestion de version

Exercice 1 :Concepts et terminologies

Q1 : Définissez les termes suivants :

Génie logiciel , Logiciel , Progiciel , Architecture logicielle, Cycle de vie du développement logiciel, U.M.L, Planification

Solution Q1 :

- **Génie logiciel** : C'est une discipline qui applique des principes scientifiques, mathématiques, et techniques pour le développement et la maintenance des logiciels. Son but est de produire des logiciels de qualité, fiables et maintenables tout en optimisant les ressources et le temps. Il a été employé pour la première fois par à la conférence de L'OTAN concernant la crise du logiciel en 1968 bien que la notion elle-même est due à la conceptrice du système embarqué du programme Apollo; l'informaticienne et mathématicienne Margaret Hamilton. Le génie logiciel désigne l'ensemble des méthodes, des techniques et d'outils concourant à la production d'un logiciel de qualité avec maîtrise des coûts et délais.
- **Logiciel** : Ensemble composé d'un ou plusieurs programmes, de procédures, ainsi que les fichiers nécessaires pour les rendre opérationnels (documents nécessaires à leur installation, utilisation, développement et maintenance., Un logiciel peut être un système d'exploitation, une application ou une bibliothèque de fonctions.
- **Progiciel** : C'est un logiciel standardisé conçu pour répondre à des besoins communs et destiné à être utilisé par un grand nombre d'entreprises ou d'utilisateurs sans modifications spécifiques. Par exemple, les ERP (Enterprise Resource Planning) sont des progiciels qui couvrent les besoins de gestion des entreprises.
- **Architecture logicielle** : C'est la structure ou la conception de haut niveau d'un logiciel. Elle définit comment les différentes parties d'un système (modules, composants) interagissent entre elles et avec l'environnement extérieur, garantissant ainsi la qualité, la maintenabilité et la scalabilité du système.
- **Cycle de vie du développement logiciel** : Il s'agit de l'ensemble des phases par lesquelles un logiciel passe depuis sa conception jusqu'à sa mise en production et sa maintenance. Les principales phases sont la collecte des exigences, la conception, le développement, le test, le déploiement et la maintenance.
- **UML (Unified Modeling Language)** : Un langage de modélisation visuelle utilisé pour représenter graphiquement la conception d'un système logiciel. UML permet de créer différents types de diagrammes (cas d'utilisation, classes, séquences, etc.) pour documenter les aspects structurels et comportementaux d'un logiciel.



TD1-Partie 1- Solution-fr

- **Planification** : En gestion de projet logiciel, la planification est l'étape qui consiste à organiser les tâches, définir les échéances, attribuer les ressources et estimer le budget pour assurer la réalisation du projet dans les délais et avec les ressources disponibles.

Q2 : Quel est la différence entre : langage, méthode et paradigme (donnez des exemples).

Solution Q2 :

Méthode = Langage + Démarche.

	Définition	Exemples
Langage	Un langage de programmation est un ensemble de règles et de syntaxe permettant aux développeurs d'écrire des instructions qu'un ordinateur peut comprendre et exécuter. Chaque langage a ses propres caractéristiques, syntaxes et domaines d'application spécifiques.	Python : Un langage de haut niveau populaire pour le développement d'applications web, l'analyse de données, et l'intelligence artificielle. Java : Connu pour sa portabilité et son utilisation dans les applications d'entreprise et les applications Android. C++ : Utilisé pour les applications de bas niveau et les jeux, apprécié pour son efficacité et son contrôle de la mémoire.
Méthode	En génie logiciel, une méthode est un ensemble de techniques, processus et pratiques qui guident le développement de logiciels, de la planification à la maintenance. C'est le processus structure la façon dont le développement logiciel doit être géré.	Méthode agile : Une approche de développement logiciel itérative et flexible qui favorise la collaboration entre les équipes et les parties prenantes. Elle repose sur des cycles de développement courts (sprints) permettant des améliorations continues et des ajustements en fonction des besoins du client. Elle est composée de méthodes spécifiques comme Scrum et Kanban. Méthode Waterfall (en cascade) : Un modèle (Processus) de développement séquentiel où chaque phase du développement logiciel (exigences, conception, développement, tests, maintenance) doit être complétée avant de passer à la suivante, sans retour en arrière. Ce modèle est rigide et convient aux projets où les exigences sont bien définies dès le départ. Méthode MERISE : Une méthode d'analyse et de conception des systèmes d'information. Il se base sur une approche structurée et séparée des données et des traitements pour modéliser les besoins d'un système d'information.
Paradigme	Un paradigme de programmation est un style ou une philosophie (comme l'orienté objet ou le fonctionnel) qui influence l'organisation et la structure du code. Les paradigmes dictent comment les problèmes doivent être abordés dans un langage donné et affectent le choix des structures de données, des	Paradigme impératif : Le programme décrit une séquence d'instructions pour changer l'état du système (ex. : C, Python). Paradigme orienté objet : Organise le code en objets contenant des données et des méthodes pour interagir avec ces données (ex. : Java, Python, C++). Paradigme fonctionnel : Privilégie l'utilisation de fonctions sans effet de bord et des expressions mathématiques (ex. : Haskell, Lisp, et, dans une certaine mesure, JavaScript avec les fonctions pures). Paradigme logique : Basé sur la logique formelle, il décrit des



TD1-Partie 1- Solution-fr

techniques et des algorithmes. Par exemple dans le contexte des langages de programmation, un paradigme fournit une manière rapide de déterminer le mode de fonctionnement du langage, et le mode de pensée à adopter quand on l'utilise (style de programmation). Ainsi, le paradigme Objet impose de penser son code sous forme d'objets travaillant ensemble, tandis le paradigme de la G.L : Est un domaine des sciences de l'ingénierie dont l'objet d'étude est la conception, la fabrication, et la maintenance des systèmes informatique complexe.	relations entre des faits et des règles (ex. : Prolog).
--	--

Q3 : En quoi consiste globalement une « Gestion de projets logiciels » ?

Solution Q3 : La **gestion de projets logiciels** comprend la planification, l'organisation, le suivi et la coordination des activités pour développer un logiciel de qualité, en respectant délais et budget.

Étapes principales :

1. **Définition des objectifs** : Recueillir et analyser les besoins fonctionnels et non fonctionnels.
2. **Planification** : Établir un plan de tâches, de calendrier et de budget.
3. **Gestion des ressources** : Assigner les rôles et organiser les outils et ressources.
4. **Suivi et contrôle** : Suivre les progrès, ajuster en cas de besoin, et gérer les risques.
5. **Assurance qualité** : Effectuer des tests pour garantir la fiabilité et la performance.
6. **Livraison et déploiement** : Préparer le logiciel pour sa mise en production.
7. **Maintenance** : Assurer le support et les mises à jour.

Q4 : What are the best software engineering techniques and methods?

Solution Q4 : While all software projects have to be professionally managed and developed, different techniques are appropriate for different types of system. For example, games should always be developed using a series of prototypes whereas safety critical control systems require a complete and analyzable specification to be developed. There are no methods and techniques that are good for everything.

Q5 : What are the key challenges facing software engineering?



TD1-Partie 1- Solution-fr

Solution Q5: Coping with increasing diversity, demands for reduced delivery times and developing trustworthy software.

Q6 : What are the costs of software engineering?

Solution Q6: Roughly 60% of software costs are development costs, 40% are testing costs. For custom software, evolution costs often exceed development costs.

Q7 : What differences has the Internet made to software engineering?

Solution Q7: Not only has the Internet led to the development of massive, highly distributed, service-based systems, it has also supported the creation of an “app” industry for mobile devices which has changed the economics of software.

Q8 : Quelles sont les qualités essentielles requises dans un cahier des charges pour un projet logiciel, ainsi que les principales raisons de réussite ou d'échec d'un projet, et comment peut-on améliorer les facteurs de qualité liés à la correction (utilité) et à la facilité d'utilisation ?

Solution Q8:

a- Un cahier des charges doit être :

- ✓ **Clair et précis** : Détails des besoins et exigences du projet.
- ✓ **Complet** : Couvre tous les aspects essentiels (fonctionnalités, contraintes, délais).
- ✓ **Conforme aux attentes** : Répond aux besoins réels des utilisateurs.
- ✓ **Évolutif** : Peut être modifié en fonction des nouvelles exigences.

b- Raisons de succès ou d'échec d'un projet :

La raison principal de la majorité des problèmes est une mauvaise appréhension des problèmes à résoudre et surtout une grave sous-estimation de leur difficulté ».

Raisons de succes d'un projet	Raisons d'échec d'un projet
Implication active des utilisateurs	Spécifications incomplètes ou ambiguës
Soutien et engagement de la hierarchie	Faible communication entre les parties prenantes
Definition claire et precise des besoins client	Mauvaise gestion des changements
Planification efficace avec suivi des delais et du budget	Sous-estimation de la complexité du projet
Communication constante entre les equipes et les parties prenantes	Manque d'implication de l'équipe ou des utilisateurs
Gestion proactive des risques et anticipation des obstacles	Absence de gestion des risques
Motivation et competences de l'equipe	Inadaptation aux exigences évolutives
Capacité d'adaptation aux changements	Résistance aux modifications nécessaires

c- Principaux facteurs de qualité d'un logiciel

- ✓ **Validité** : Remplir les fonctions spécifiées.
- ✓ **Fiabilité** : Fonctionnement correct en conditions anormales.
- ✓ **Facilité d'utilisation** : Interface intuitive et prise en main simple.



TD1-Partie 1- Solution-fr

- ✓ **Compatibilité** : Interaction harmonieuse avec d'autres systèmes.
 - ✓ **Efficacité** : Utilisation optimale des ressources.
 - ✓ **Portabilité** : Fonctionnement sur différentes plateformes.
 - ✓ **Réutilisabilité** : Utilisation du code pour d'autres projets.
 - ✓ **Maintenabilité** : Facilité de modification et d'amélioration.
 - ✓ **Extensibilité** : Adaptation aux changements de spécifications.
 - ✓ **Intégrité** : Protection des données et du code.
 - ✓ **Ponctualité** : Livraison dans les délais.
- d- Amélioration des facteurs de qualité
- **Correction (Utilité)** :
 - ✓ Effectuer une analyse rigoureuse des besoins en amont.
 - ✓ Améliorer la communication (langage commun, démarche participative).
 - ✓ Travailler avec méthode et rigueur pour minimiser les erreurs.
 - **Facilité d'utilisation** :
 - ✓ Analyser les modes opératoires des utilisateurs pour adapter le logiciel.
 - ✓ Concevoir une interface utilisateur (IHM) ergonomique pour faciliter la navigation et la prise en main.

Q9: Comment peut-on réduire l'écart entre les besoins réels d'un client et les besoins exprimés dans le cahier de charges ?

Solution Q9: Pour réduire l'écart entre les besoins réels et ceux exprimés dans le cahier de charges :

- ✓ **Analyse approfondie** : Organiser des ateliers et interviews pour clarifier les besoins.
- ✓ **Implication des utilisateurs finaux** : Associer les utilisateurs dès le début pour mieux cerner leurs attentes.
- ✓ **Validation continue** : Utiliser des prototypes et valider régulièrement chaque étape avec le client.
- ✓ **Communication régulière** : Maintenir une communication claire et éviter le jargon technique.
- ✓ **Méthodes agiles** : Adopter des cycles courts pour intégrer les retours client et ajuster les spécifications.

Q10 : Opérer un distinguo entre « cycle de vie » et « cycle de développement » du logiciel

Solution Q10 :

Le **cycle de vie du logiciel** englobe toutes les étapes, depuis la conception initiale jusqu'à la fin de vie du logiciel. Il inclut des phases telles que l'analyse des besoins, la conception, le développement, la maintenance et le retrait. L'objectif principal de ce cycle est d'assurer la qualité et l'évolution continue du logiciel tout au long de sa durée de vie. En revanche,

le **cycle de développement du logiciel** se concentre spécifiquement sur la création d'une version initiale ou d'une mise à jour du logiciel. Il comprend des étapes comme l'analyse, la conception, le codage, les tests et la livraison, avec pour objectif de produire une version fonctionnelle. Ainsi, le cycle de développement est une partie intégrante du cycle de vie, qui est plus global et inclut également la maintenance et l'évolution du logiciel.



TD1-Partie 1- Solution-fr

Q11 : Quelle sont les principaux modèles d'un Cycle de Vie d'un logiciel ?

Solution Q11 :

Modèle	Caractéristiques	Avantages	Inconvénients
Modèle en cascade	Phases séquentielles : conception, développement, tests, déploiement, maintenance.	Simple à comprendre et à gérer.	Rigide, difficile à revenir en arrière une fois une phase terminée.
Modèle itératif	Développement en cycles, permettant des révisions et des améliorations progressives.	Flexibilité pour intégrer des changements.	Peut être difficile à gérer si les itérations ne sont pas bien définies.
Modèle agile	Développement par petites étapes avec des cycles courts et des retours fréquents.	Réaction rapide aux changements, forte collaboration.	Nécessite une communication constante et une implication des parties prenantes.
Modèle en V	Similaire au modèle en cascade, mais avec des phases de test parallèles.	Tests systématiques et validation à chaque phase.	Complexité dans la gestion des phases de test et de développement.
Modèle en spirale	Combinaison de développement itératif et de gestion des risques, avec des phases de planification.	Approche centrée sur le risque, flexibilité dans les changements.	Peut être coûteux en termes de temps et de ressources.
Modèle DevOps	Intégration du développement et des opérations pour améliorer le cycle de livraison.	Déploiements plus fréquents et réactivité aux besoins.	Peut nécessiter des changements culturels importants au sein de l'équipe.

Q12 : Les modèles de cycle de vie classiques ou modernes décrivent tous, la démarche à suivre pour produire un logiciel de qualité. Cependant, il existe une caractéristique qui les différencie. Laquelle ? et quelles sont les limites des modèles de cycle de vie classiques ? Quels sont les apports des méthodes modernes telles que Agile et DevOps.

Solution 12 :

Les modèles de cycle de vie classiques et modernes décrivent une démarche de production logicielle, mais se distinguent principalement par leur **souplesse face aux changements**. Les modèles classiques,



TD1-Partie 1- Solution-fr

comme le modèle en cascade, sont rigides et suivent une structure séquentielle où chaque étape doit être finalisée avant de passer à la suivante. En revanche, les méthodes modernes telles que **Agile** et **DevOps** privilégient des processus itératifs et adaptatifs, permettant d'intégrer les modifications en cours de projet.

Limites des modèles classiques de cycle de vie :

1. **Rigidité** : Les modèles classiques, comme le modèle en cascade, ne permettent pas de revenir facilement en arrière si des changements sont nécessaires.
2. **Manque de flexibilité face aux imprévus** : Une fois les spécifications définies, il devient complexe de les ajuster.
3. **Long délai avant la livraison** : La production d'une version exploitable du logiciel est souvent longue, ce qui peut décevoir le client s'il ne voit pas de résultats intermédiaires.
4. **Faible interaction avec le client** : Le client n'intervient que peu, ce qui peut entraîner un écart entre le produit final et les attentes réelles.

Apports des méthodes modernes (Agile et DevOps) :

1. **Adaptabilité et itération** : Les cycles de développement courts permettent des ajustements rapides en fonction des retours du client.
2. **Livraison continue** : En DevOps, les logiciels sont livrés et déployés de façon continue, permettant d'intégrer rapidement des améliorations.
3. **Collaboration accrue** : Une forte interaction entre les équipes (développement, opérations et client), améliore la compréhension des besoins.
4. **Amélioration de la qualité** : Avec des tests fréquents et un suivi en temps réel, les défauts sont détectés et corrigés plus tôt.

Q13 : Comparer le prototypage jetable par rapport au prototypage non jetable.

Solution 13 :

Aspect	Prototypage jetable	Prototypage non jetable
Objectif	Tester des idées ou concepts rapidement sans intention de réutilisation.	Créer une version initiale qui évoluera vers le produit final.
Utilisation	Conçu pour être jeté après validation des exigences.	Conçu pour être amélioré et intégré dans le logiciel final.
Qualité du code	Code souvent de moindre qualité, axé sur la rapidité de création.	Code de qualité qui respecte les standards de développement.
Temps de développement	Rapidité de développement, car le prototype est temporaire.	Peut être plus long car le prototype est développé pour durer.
Avantage	Permet des itérations rapides, facile à réaliser.	Économie de ressources, car le code est réutilisable.
Inconvénient	Nécessite de recommencer le développement pour le produit final.	Demande une planification et un développement initial plus rigoureux.

Q14 : Comment sont décrites les activités d'un processus de développement.



TD1-Partie 1- Solution-fr

Solution : Les activités d'un processus de développement sont décrites en termes de :

- a. Entrées de l'activité (matière première) ;
- b. Sortie de l'activité (résultat) ;
- c. Intervenant et rôles (qui ?) ;
- d. Description de l'activité (quoi – quel est le problème à traiter ?) ;
- e. Standards, guides, « best practices » à appliquer (comment ?).

Q15 : Citer les étapes de développement d'un logiciel en expliquant chaque étape.

Solution : les étapes essentielles du développement logiciel :

1. **Analyse des besoins** : Identification et documentation des attentes et contraintes des utilisateurs.
2. **Spécifications** : Formalisation des fonctionnalités et exigences techniques dans un document de référence.
3. **Conception** : Planification de l'architecture du logiciel pour structurer ses composants.
4. **Développement** : Programmation des fonctionnalités selon les spécifications, incluant des tests unitaires.
5. **Tests et validation** : Vérification du logiciel pour garantir sa conformité aux spécifications et son bon fonctionnement.
6. **Déploiement** : Installation du logiciel pour les utilisateurs finaux avec une éventuelle formation.
7. **Maintenance** : Corrections et mises à jour pour adapter le logiciel aux besoins évolutifs.